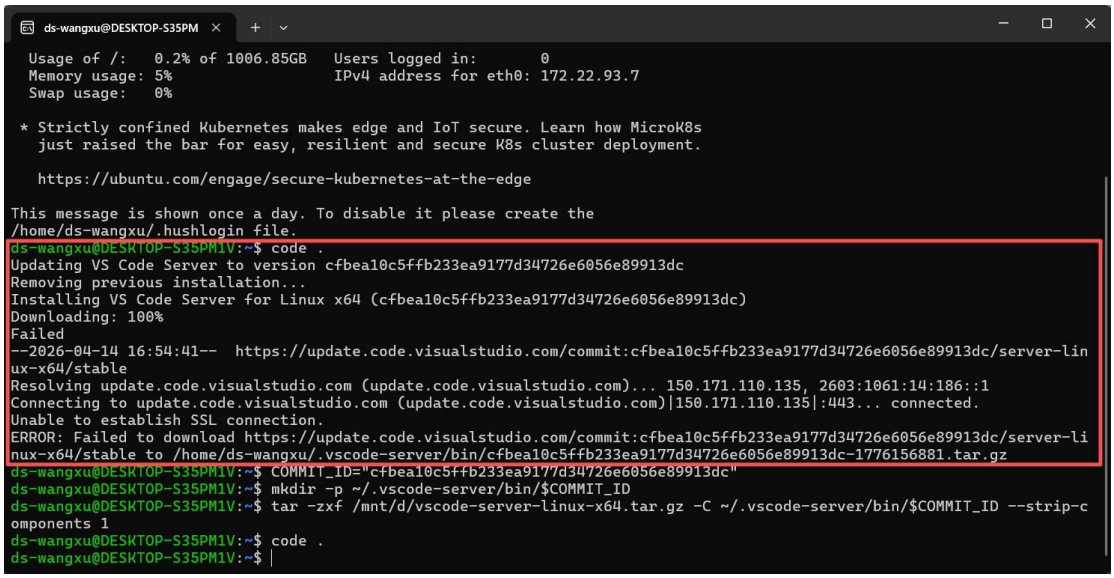
一、前言：

按理说，在我们启动目标版本的 Ubuntu 系统之后，在对应的命令行界面，直接输入 code .即可，WSL 内部会自动安装对应的 VSCode 服务器端程序。

但是，由于最近(今天是 2026.4.14)，**国家的防火墙(疑似)升级了**，现在(似乎)更严了，因此可能影响到了部分向外网的连接，所以才导致了我们在前天遇到的这个问题，好在经过两天的调试，我们终于找到了问题所在。

二、遇到的问题描述：




```
ds-wangxu@DESKTOP-S35PM1V:~$ code .
Updating VS Code Server to version cfbea10c5ffb233ea9177d34726e6056e89913dc
Removing previous installation...
Installing VS Code Server for Linux x64 (cfbea10c5ffb233ea9177d34726e6056e89913dc)
Downloading: 100%
Failed
--2026-04-14 16:54:41--
https://update.code.visualstudio.com/commit:cfbea10c5ffb233ea9177d34726e6056e89913dc/server-linux-x64/stable
Resolving update.code.visualstudio.com (update.code.visualstudio.com)... 150.171.110.135, 2603:1061:14:186::1
Connecting to update.code.visualstudio.com (update.code.visualstudio.com)|150.171.110.135|:443... connected.
Unable to establish SSL connection.
ERROR: Failed to download https://update.code.visualstudio.com/commit:cfbea10c5ffb233ea9177d34726e6056e89913dc/server-linux-x64/stable to
```

```
/home/ds-wangxu/.vscode-server/bin/cfbea10c5ffb233ea9177d34726e
6056e89913dc-1776156881.tar.gz
```

三、问题基本背景介绍：

1. VS Code 远程开发的原理：

当我们在 WSL 的终端里输入 code .时，实际上启动了一个“远程开发”流程。这个过程需要两个部分协同工作：

(1) 客户端：我们安装在 Windows 上的 VS Code 主程序。它负责提供用户界面（窗口、菜单、编辑器）

(2) 服务器端：一个需要运行在 WSL（Linux 环境）里的轻量级程序（VS Code Server）。它负责在 Linux 内部执行命令、管理文件、运行扩展等。

为确保 VS Code 客户端（Windows 端）与服务器端（WSL 端）之间的绝对兼容性，VS Code 设计为：

当客户端启动时，它会检查 WSL 中安装的服务器版本是否与自身兼容。如果发现版本不符，它会强制下载并安装一个与自身完全匹配的正确版本。这就是为什么每次自动更新失败时，它都执着地尝试去下载那个特定哈希值对应的文件。

2. 自动安装如何工作：

VS Code 客户端通过一个固定的网址格式，从微软的官方服务器下载这个服务器程序的压缩包。这个网址的格式如同我们在”问题描述”中所展现的：https://update.code.visualstudio.com/commit:版本号/server-linux-x64/stable。

3. SSL/TLS 是什么：

https://开头的网址，其中的 s 代表“安全”（Secure）。它意味着我们的电脑（WSL）和服务器之间的通信是加密的，防止被窃听或篡改。建立这种加密连接的过程，就叫做“SSL/TLS 握手”。

四、问题根源——为什么会出现 SSL 连接失败：

在我们的错误信息中，可以看到始终存在这样一个问题：

> Connecting to update.code.visualstudio.com... **connected**.
>
> **Unable to establish SSL connection.**

这告诉我们：

第一步（网络连接）成功：我们的 WSL 能够找到服务器并在两者之间建立一条基础的、未加密的“数据通道”（TCP 连接）。这就像我们拨通了对方的电话，对方“喂”了一声。

第二步（安全握手）失败：当双方尝试为这条通道加上“加密锁”（SSL/TLS 握手）时，握手过程被异常终止了。错误信息 Connection reset by peer（对端重置连接）就像电话突然被对方挂断。

那么是谁”挂断”了电话呢？

我们的 WSL 本身是正常的，因为我们用 sudo apt update 成功从 Ubuntu 官方服务器下载了信息，这同样使用了 HTTPS。这说明我们的 WSL 具备正常的 HTTPS 通信能力。

因此，最可能的情况是，在我们的 WSL 到 update.code.visualstudio.com 服务

器的网络路径上，存在一个“**中间人**”。这个“中间人”通常是**公司防火墙/网络安全设备**，或者**网络代理或特定安全软件**。结合我们最近了解到，似乎国家的墙加强了，似乎更严，因此可能是受到了一定的影响。

五、解决方法：

我们的解决方案核心思想是：**既然“自动打电话下载”这个环节被拦截，那我们就“离线安装”**。也就是说，既然自动下载路径被网络政策阻断，那么我们就在 Windows 环境中首先手动去那个网站下载，并将这个安装包解压到 VS Code 预期的准确位置。这样当 VS Code 启动时，会发现所需文件已经存在，就不需要再去通过有问题的网络来下载，从而成功连接。

1.设置 Commit ID 变量：

```
COMMIT_ID="cfbea10c5ffb233ea9177d34726e6056e89913dc"
```

设置了一个变量，把复杂的版本号存起来，方便后面使用，避免重复输入出错。这个版本号必须和 VS Code 客户端当前需要的版本完全一致，也就是我们在错误信息中输出过的所需要的版本号。

2.创建存放 VS Code Server 的目录：

```
mkdir -p ~/.vscode-server/bin/$COMMIT_ID
```

在 WSL 的家目录(~)下，创建 VS Code 存放服务器程序的固定位置。-p 参数确保如果中间目录不存在也会一并创建。VS Code 客户端会固定来这里寻找指定版本的服务器文件。(同样见于错误信息输出的目的目录)

3.解压 Windows 下载目录中的文件到刚才创建的目录中：

```
tar -zxf /mnt/d/vscode-server-linux-x64.tar.gz -C ~/.vscode-server/bin/$COMMIT_ID
--strip-components 1
```

tar -zxf：这是 Linux 的解压命令，用于解压.tar.gz 格式的压缩包。

/mnt/d/vscode-server-linux-x64.tar.gz：这是压缩包在 WSL 中的路径。因为 WSL 可以直接访问 Windows 的磁盘，/mnt/d/对应我们 Windows 的 D 盘。我们已经在 Windows 上用浏览器（挂梯子）成功下载了文件。

-C ~/.vscode-server/bin/$COMMIT_ID：指定解压的目标目录，就是上一步创建的那个特定版本目录。

--strip-components 1：这是一个精妙的参数。压缩包内部通常最外层还有一个文件夹。这个参数的作用是“剥掉”最外层的这一级目录，直接将内部的所有文件解压到目标目录下。这确保了 VS Code 打开目录时，直接就能找到可执行文件，而不是藏在又多一层文件夹里。

4.再次运行 code .：

```
code .
```

VS Code Windows 客户端启动，照例先去检查 WSL 中的 ~/.vscode-server/bin/cfbea10c5ffb.../目录。这一次，它发现所需要的服务器文件已经存在了，于是直接使用现成的文件启动服务器端程序。随后，客户端与服务器端成功建立连接，远程开发窗口就顺利打开了。

六、关于未来：

我们可能还会关心，如果以后 VS Code 升级了怎么办？

1. 如果升级不要求强制更换 WSL 服务器版本，那一切照旧，没问题。
2. 如果升级要求新版本的服务器，当我们再次运行 code .时，过程会重演：客户端尝试下载新版本 -> 再次被网络拦截而失败。

到时候我们的应对方法：重复一次这个“手动安装”流程即可。只需从新的错误信息中**提取新的版本号**，用新版本号去构造**下载链接**、**创建目录**和解压即可。